

Experiment 5

Dynamic React Form

App.js

```
// Program 5: Dynamic form generated from a JavaScript configuration object
// No extra packages needed. Usage: save as src/App.jsx
import React, { useState } from "react";

// ----- Configuration object -----
const formConfig = {
  title: "Student Registration",
  fields: [
    { name: "fullName", label: "Full Name", type: "text", placeholder: "Enter name", required: true },
    { name: "email", label: "Email", type: "email", placeholder: "Enter email", required: true },
    { name: "age", label: "Age", type: "number", placeholder: "Enter age" },
    { name: "gender", label: "Gender", type: "radio", options: ["Male", "Female", "Other"] },
    { name: "course", label: "Course", type: "select", options: ["BCA", "B.Tech", "MCA", "MBA"] },
    { name: "skills", label: "Skills", type: "checkbox", options: ["React", "Java", "Python"] },
    { name: "about", label: "About You", type: "text", placeholder: "Short description" },
  ],
};

// Build initial state from config
const buildInitialState = (config) => {
  const state = {};
  config.fields.forEach((f) => {
    state[f.name] = f.type === "checkbox" ? [] : "";
  });
  return state;
};

export default function App() {
  const [formData, setFormData] = useState(buildInitialState(formConfig));
  const [submitted, setSubmitted] = useState(null);

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    if (type === "checkbox") {
      setFormData((prev) => ({
        ...prev,
        [name]: checked ? [...prev[name], value] : prev[name].filter((v) => v !== value),
      }));
    } else {
      setFormData((prev) => ({ ...prev, [name]: value }));
    }
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmitted(formData);
  };

  // Render one field depending on its type
  const renderField = (f) => {
    switch (f.type) {
      case "select":
        return (
          <select name={f.name} value={formData[f.name]} onChange={handleChange} required={f.required}>
            <option value="">-- Select --</option>
            {f.options.map((o) => <option key={o} value={o}>{o}</option>)}
          </select>
        );
      case "radio":
        return f.options.map((o) => (
          <label key={o} style={{ marginRight: 12 }}>
            <input type="radio" name={f.name} value={o}
              checked={formData[f.name] === o} onChange={handleChange} /> {o}
          </label>
        ));
      case "checkbox":
        return f.options.map((o) => (
          <label key={o} style={{ marginRight: 12 }}>
            <input type="checkbox" name={f.name} value={o}
              checked={formData[f.name].includes(o)} onChange={handleChange} /> {o}
          </label>
        ));
      case "text":
        return <input type="text" name={f.name} placeholder={f.placeholder} />
    }
  };
}
```

Experiment 5

Dynamic React Form

App.js (Contd.)

```
        value={formData[f.name]} onChange={handleChange} />;
    default:
      return <input type={f.type} name={f.name} placeholder={f.placeholder}
        value={formData[f.name]} onChange={handleChange} required={f.required} />;
    }
  };

  return (
    <div style={{ maxWidth: 480, margin: "30px auto", fontFamily: "sans-serif" }}>
      <h2>{formConfig.title}</h2>
      <form onSubmit={handleSubmit}>
        {formConfig.fields.map((f) => (
          <div key={f.name} style={{ marginBottom: 14 }}>
            <label style={{ display: "block", fontWeight: "bold", marginBottom: 4 }}>
              {f.label}{f.required && " *"}
            </label>
            {renderField(f)}
          </div>
        ))}
        <button type="submit">Submit</button>
      </form>

      {submitted && (
        <div style={{ marginTop: 25, padding: 12, background: "#f3f3f3" }}>
          <h3>Submitted Data</h3>
          {formConfig.fields.map((f) => (
            <p key={f.name}>
              <b>{f.label}</b>: <b>{" "}
                {Array.isArray(submitted[f.name]) ? submitted[f.name].join(", ") : submitted[f.name]}
              </b>
            </p>
          ))}
        </div>
      )}
    </div>
  );
}
```

Experiment 5

Output

Student Registration

Full Name *

Enter name

Email *

Enter email

Age

Enter age

Gender

☐ Male ☐ Female ☐ Other

Course

-- Select --

Skills

☐ React ☐ Java ☐ Python

About You

Short description

Submit

Experiment 5

Dynamic React Form

Aim

1. To create a React form whose fields are dynamically generated from a JavaScript configuration object.
2. To store user input using state and display submitted data.

Procedure

1. Create a JavaScript configuration object for the form fields.
2. Generate the fields dynamically from the configuration.
3. Store values using React state and handle changes.
4. Submit the form and display the submitted data.

Sample Input

- Enter student details, choose options and click Submit.

Sample Output

- The dynamic Student Registration form is displayed and submitted values can be shown below it.

Result

Thus, a dynamic React form was successfully generated from a JavaScript configuration object using state.